

F1 PIT STOP PREDICTION

Preprocessing Pipeline — Design & Implementation Report

Kaggle Playground Series S6E5

Module: `f1_preprocessor.py`

1 Project Overview

This report documents the complete preprocessing pipeline designed for the F1 Pit Stop Prediction machine learning project (Kaggle Playground Series S6E5). The pipeline operates downstream of the custom feature engineering transformer (F1PitStopFeatureTransformer) and upstream of the final model ensemble.

1.1 Task Definition

- Problem type:** Binary classification
- Target variable:** PitNextLap — will the driver pit on the next lap? (0 or 1)
- Class distribution:** 80.1% negative (no pit) / 19.9% positive (pit) — moderately imbalanced
- Evaluation:** ROC-AUC / F1 (implied by binary competition format)

1.2 Dataset Characteristics (train.csv)

Property	Value	Notes
Total rows	439,140	All laps across 4 seasons
Raw columns	16	Including target
Years covered	2022 – 2025	4 F1 seasons
Missing values	Zero (0)	No imputation required for raw data
Duplicates	Zero (0)	Confirmed via EDA
LapTime outliers >130s	3,193 rows (0.73%)	SC / red-flag laps
Cumulative_Degradation negatives	375,742 rows (85.6%)	Pit lap resets degrade to negative

1.3 Compound Distribution

Compound	Count	Share
MEDIUM	211,141	48.1%
HARD	170,518	38.8%
SOFT	38,744	8.8%
INTERMEDIATE	17,382	4.0%
WET	1,355	0.3%

2 Feature Engineering Summary (F1PitStopFeatureTransformer)

The custom transformer produces 79 features from the 15 raw input columns. These are grouped into eight engineering modules, each targeting a specific domain of pit-stop decision-making.

Module	# Features	Description
A — Tyre Degradation	14	Polynomial tyre life terms, compound cliff lag, overrun amount, degradation rate and acceleration, compound softness ranking
B — Lap Time Performance	3	Z-score vs race baseline, absolute delta, worsening flag
C — Rolling / Lag Temporal	16	3-window rolling means/stds for LapTime and delta, degradation rolling means, lag-1 lap time, 3-lap pace momentum, consecutive-worsening run-length, prev pitstop flag
D — Race Strategy & Position	12	Laps remaining (raw + normalised), too-late-to-pit flag, undercut/overcut windows, position normalised, rolling position trend, stint exhaustion
E — Compound One-Hot	6	5 binary compound dummies + compound × tyre-age interaction
F — Race Phase & Stint	6	Early/mid/late phase bins, Pirelli pit window flag, first-stint flag, laps past expected stint
G — Key Interactions	4	Tyre age × race progress, urgency index, cliff × window, momentum × overrun
H — Safety Car Proxy	5	SC lap proxy, pit cluster size, cluster lap flag, recent SC rolling sum, SC spike flag
Encodings	2	Ordinal driver + race encodings (LabelEncoder, -1 for unseen)
Raw Pass-Through	11	Year, PitStop, LapNumber, Stint, TyreLife, Position, LapTime(s), LapTime_Delta, Cumulative_Degradation, RaceProgress, Position_Change

Note: All NaN values introduced by rolling/lag operations (first lap of each stint has no history) are forward-filled then zero-filled inside the transformer. The preprocessing module therefore operates on a fully-dense numeric matrix.

3 Preprocessing Design Rationale

The preprocessing pipeline operates on the 79-column engineered feature matrix. Features are partitioned into five groups based on their statistical character and semantic meaning. Each group receives a tailored transformation chain.

3.1 Why Separate Groups?

A single global scaler (e.g. StandardScaler applied uniformly) would be incorrect for this matrix for three reasons:

- Binary flags (0/1) already have a natural scale; scaling them creates fractional values that confuse tree-based models and add no information for linear models.
- High-skew continuous features (LapTime, Cumulative_Degradation, urgency_index) have asymmetric distributions with heavy tails. StandardScaler on a skewed feature still produces a non-Gaussian distribution, which degrades gradient-based models. A power transform normalises the distribution first.
- Bounded-ratio features (RaceProgress, tyre_life_norm, laps_remaining_norm) are already in [0, 1] and carry direct physical meaning. MinMaxScaler preserves this semantic while ensuring values do not exceed the training range on unseen data.

3.2 Feature Group Assignments

Group 1 — Binary Flags (23 features)

Columns: PitStop, past_cliff, is_wet_compound, is_soft_compound, laptime_worsening, prev_pitstop, too_late_to_pit, undercut_window, overcut_candidate, in_points, cpd_HARD, cpd_INTERMEDIATE, cpd_MEDIUM, cpd_SOFT, cpd_WET, race_phase_early, race_phase_mid, race_phase_late, in_pit_window, is_first_stint, sc_lap_proxy, is_cluster_lap, sc_spike_flag.

Transformation: SafeImputer (most_frequent, defensive) → passthrough

Rationale: These are already 0/1 integers. Tree ensembles (XGBoost, LightGBM, RandomForest) require no normalisation. For gradient/linear models, pass passthrough_binary=False to apply StandardScaler.

Group 2 — Ordinal Integers (15 features)

Columns: Year, LapNumber, Stint, TyreLife, Position, compound_softness, compound_expected_stint, compound_cliff_lap, total_race_laps, laps_remaining, consecutive_worse, pit_cluster_size, recent_sc_proxy, driver_enc, race_enc.

Transformation: SafeImputer (median) → StandardScaler

Rationale: These are integer counts with moderate ranges. StandardScaler centres them at zero, which helps gradient-boosting models reduce split-threshold sensitivity. driver_enc and race_enc are ordinal but included here since their integer range (0–100+) can dominate split calculations in non-tree models if unscaled.

Group 3 — High-Skew Continuous (20 features)

Columns: LapTime(s), LapTime_Delta, Cumulative_Degradation, laptime_zscore, laptime_delta_abs, tyre_life_sq, deg_per_lap, deg_accel, urgency_index, cliff_proximity, tyreage_x_progress, momentum_x_overrun, pace_momentum_3, roll_lt_std_1/3/5, laptime_lag1, roll_lt_mean_1/3/5.

Transformation: SafeImputer (median) → WinsorCapper (0.1%, 99.9%) → PowerTransformer (Yeo-Johnson) → StandardScaler

Why WinsorCapper first: LapTime has confirmed outliers > 130 s (SC laps).

Cumulative_Degradation has large negatives from pit lap resets. PowerTransformer is sensitive to extreme outliers — Winsorization at the 0.1% / 99.9% quantile boundary removes the ~3,200 SC-lap outliers without discarding the rows.

Why Yeo-Johnson over Box-Cox: Box-Cox requires strictly positive input. Cumulative_Degradation and LapTime_Delta contain negatives and zeros — Yeo-Johnson handles all real values.

Why StandardScaler after PowerTransformer: PowerTransformer does not guarantee unit variance. A follow-up StandardScaler ensures all features are on the same scale for gradient-based learners.

Group 4 — Bounded-Ratio Features (5 features)

Columns: RaceProgress, laps_remaining_norm, position_norm, tyre_life_norm, tyre_overrun_amount.

Transformation: SafeImputer (median) → MinMaxScaler(0, 1)

Rationale: These are already [0, 1] fractions by construction. MinMaxScaler preserves the bounded nature and clips test-set values that might slightly exceed training bounds due to unseen races with more laps than training data. No power transform needed since the distributions are already near-uniform.

Group 5 — Standard Continuous (16 features)

Columns: tyre_life_log, laps_to_cliff, laps_past_expected, stint_exhaustion, cpd_rank_x_life, position_change_abs, position_trend, roll_pos_mean_3, roll_delta_1/3/5, roll_deg_1/3/5, cliff_x_window, Position_Change.

Transformation: SafeImputer (median) → RobustScaler

Why RobustScaler over StandardScaler here: These features have moderate skew and are influenced by positional events (safety cars, overtakes) that create legitimate but infrequent extreme values. RobustScaler uses median and IQR, making it naturally robust to the remaining outliers without explicitly capping them. This group is a step below Group 3 in skew severity but above the ordinal integers.

4 Pipeline Architecture

4.1 Class Design

F1Preprocessor inherits from sklearn's BaseEstimator and TransformerMixin, making it fully compatible with Pipeline, GridSearchCV, and cross-validation wrappers. The fit/transform API exactly mirrors F1PitStopFeatureTransformer.

4.2 Data Flow Diagram

```
x_train (raw, 15 cols, 395,226 rows)
└─▶ F1PitStopFeatureTransformer.fit_transform()
    [Sort by Race/Year/Driver/LapNumber]
    [A] Tyre degradation features (14)
    [B] Lap time performance features (3)
    [C] Rolling/lag temporal features (16)
    [D] Race strategy & position features (12)
    [E] Compound one-hot + interactions (6)
    [F] Race phase & stint features (6)
    [G] Key cross-feature interactions (4)
    [H] Safety car proxy features (5)
    [Enc] Driver + race ordinal encodings (2)
    [ffill → zero-fill NaNs]

↓

x_train_fe (engineered, 79 cols)
└─▶ F1Preprocessor.fit_transform()
    Binary flags (23) → SafeImpute → Passthrough
    Ordinal ints (15) → SafeImpute → StandardScaler
    High-skew cont (20) → SafeImpute → Winsor → YeoJohnson →
StandardScaler
    Bounded ratio (5) → SafeImpute → MinMaxScaler
    Std cont (16) → SafeImpute → RobustScaler

↓

x_train_pp (final, 79 cols) —▶ Feature Selection —▶ Model
Ensemble
```

4.3 Import & Usage (in main.ipynb)

```
# — Cell: Import preprocessor —————
from f1_preprocessor import F1Preprocessor, build_pipeline

# — Cell: Fit on train, transform both splits —————
preprocessor = F1Preprocessor() # default:
passthrough binary
x_train_pp = preprocessor.fit_transform(x_train_fe) # fit on train only
x_test_pp = preprocessor.transform(x_test_fe) # use train stats

# — Cell: Sanity check —————
print(x_train_pp.shape) # (395226, 79)
```

```

print(x_train_pp.isnull().sum().sum())      # 0
print(preprocessor.get_feature_groups())    # group assignments

# — Optional: combined pipeline —
from f1_feature_transformer import F1PitStopFeatureTransformer
pipe = build_pipeline(F1PitStopFeatureTransformer(lag_windows=[1,3,5]))
x_ready = pipe.fit_transform(x_train)

```

5 Complete Feature Inventory

The table below maps all 79 output features to their assigned preprocessing group and transformation chain.

Feature(s)	Group	Transformation	Notes
Year	Ordinal Int	StandardScaler	2022–2025; temporal trend
PitStop	Binary Flag	Passthrough	Did driver pit this lap?
LapNumber	Ordinal Int	StandardScaler	Lap count within race
Stint	Ordinal Int	StandardScaler	Stint number (1–4 typical)
TyreLife	Ordinal Int	StandardScaler	Laps on current tyre
Position	Ordinal Int	StandardScaler	Race position
LapTime (s)	High-Skew	Winsor → YeoJohnson → Std	Raw lap time; skew ~80
LapTime_Delta	High-Skew	Winsor → YeoJohnson → Std	Change vs prior lap; heavy tails
Cumulative_Degradation	High-Skew	Winsor → YeoJohnson → Std	Neg at pit reset; skew ~3.2
RaceProgress	Bounded Ratio	MinMaxScaler	[0,1] fraction of race done
Position_Change	Std Cont	RobustScaler	Signed position change
tyre_life_sq	High-Skew	Winsor → YeoJohnson → Std	TyreLife ² — nonlinear wear
tyre_life_log	Std Cont	RobustScaler	log1p(TyreLife) — compresses extremes
compound_expected_stint	Ordinal Int	StandardScaler	Reference stint length per compound
tyre_life_norm	Bounded Ratio	MinMaxScaler	TyreLife / expected; [0, 1+]
tyre_overrun_amount	Bounded Ratio	MinMaxScaler	(tyre_life_norm - 1).clip(0)

Feature(s)	Group	Transformation	Notes
compound_cliff_lap	Ordinal Int	StandardScaler	Lap where deg accelerates
laps_to_cliff	Std Cont	RobustScaler	Signed distance to cliff lap
past_cliff	Binary Flag	Passthrough	1 if TyreLife > cliff lap
cliff_proximity	High-Skew	Winsor → YeoJohnson → Std	$\exp(-0.15 * \text{laps_to_cliff})$; nonlinear
deg_per_lap	High-Skew	Winsor → YeoJohnson → Std	Cumul_Deg / TyreLife; right-skewed
deg_accel	High-Skew	Winsor → YeoJohnson → Std	Deg / TyreLife ² ; very right-skewed
compound_softness	Ordinal Int	StandardScaler	1(SOFT)–5(WET) rank
is_wet_compound	Binary Flag	Passthrough	INTERMEDIATE or WET
is_soft_compound	Binary Flag	Passthrough	SOFT compound
laptime_zscore	High-Skew	Winsor → YeoJohnson → Std	Z-score vs race mean; heavy tails
laptime_delta_abs	High-Skew	Winsor → YeoJohnson → Std	LapTime_Delta ; right-skewed
laptime_worsening	Binary Flag	Passthrough	LapTime_Delta > 0.25s
roll_lt_mean_1/3/5	High-Skew	Winsor → YeoJohnson → Std	Rolling LapTime mean (3 windows)
roll_lt_std_1/3/5	High-Skew	Winsor → YeoJohnson → Std	Rolling LapTime std (3 windows)
roll_delta_1/3/5	Std Cont	RobustScaler	Rolling LapTime_Delta mean
roll_deg_1/3/5	Std Cont	RobustScaler	Rolling Cumul_Deg mean
laptime_lag1	High-Skew	Winsor → YeoJohnson → Std	Lag-1 lap time
pace_momentum_3	High-Skew	Winsor → YeoJohnson → Std	3-lap pace slope; heavy tails
consecutive_worse	Ordinal Int	StandardScaler	Run-length of worsening laps
prev_pitstop	Binary Flag	Passthrough	1 if pitted previous lap
total_race_laps	Ordinal Int	StandardScaler	Total laps in race
laps_remaining	Ordinal Int	StandardScaler	Laps left in race
laps_remaining_norm	Bounded Ratio	MinMaxScaler	[0,1] fraction remaining
too_late_to_pit	Binary Flag	Passthrough	laps_remaining ≤ 5

Feature(s)	Group	Transformation	Notes
undercut_window	Binary Flag	Passthrough	Undercut strategy viable
overcut_candidate	Binary Flag	Passthrough	Overcut strategy viable
position_norm	Bounded Ratio	MinMaxScaler	Position / 20; [0.05–1]
position_change_abs	Std Cont	RobustScaler	Position_Change
in_points	Binary Flag	Passthrough	Position <= 10
roll_pos_mean_3	Std Cont	RobustScaler	Rolling position mean
position_trend	Std Cont	RobustScaler	Current pos - rolling mean
stint_exhaustion	Std Cont	RobustScaler	tyre_life_norm × compound_softness
cpd_HARD/ INTERMEDIATE/MEDIUM/ SOFT/WET	Binary Flag	Passthrough	Compound one-hot dummies
cpd_rank_x_life	Std Cont	RobustScaler	compound_softness × tyre_life_norm
race_phase_early/mid/late	Binary Flag	Passthrough	Race-phase one-hot
in_pit_window	Binary Flag	Passthrough	Pirelli primary pit window
is_first_stint	Binary Flag	Passthrough	Stint == 1
laps_past_expected	Std Cont	RobustScaler	(TyreLife - expected).clip(0)
tyreage_x_progress	High-Skew	Winsor → YeoJohnson → Std	tyre_life_norm × RaceProgress
urgency_index	High-Skew	Winsor → YeoJohnson → Std	tyre_life_norm / laps_rem_norm; spiky
cliff_x_window	Std Cont	RobustScaler	cliff_proximity × in_pit_window
momentum_x_overrun	High-Skew	Winsor → YeoJohnson → Std	pace_momentum × overrun_flag
sc_lap_proxy	Binary Flag	Passthrough	laptime_zscore > 0.8
pit_cluster_size	Ordinal Int	StandardScaler	Drivers pitting same lap
is_cluster_lap	Binary Flag	Passthrough	pit_cluster_size >= 3
recent_sc_proxy	Ordinal Int	StandardScaler	Rolling SC spike count (0–3)
sc_spike_flag	Binary Flag	Passthrough	laptime_zscore > 0.6
driver_enc	Ordinal Int	StandardScaler	Label-encoded driver ID
race_enc	Ordinal Int	StandardScaler	Label-encoded race ID

6 Outlier & Anomaly Handling

6.1 LapTime Outliers (Safety Car / Red Flag Laps)

3,193 rows (0.73%) have LapTime > 130 s — these are SC, VSC, or red-flag laps where the field crawls behind a safety car. These are NOT noise; they carry genuine strategic signal (drivers often pit under SC). They must not be removed.

Strategy: WinsorCapper clips LapTime at the 99.9th-percentile (approx. 105–110 s depending on circuit) before the PowerTransformer. The values are compressed into a less extreme range while the lap's presence and relative ordering are preserved.

Additionally, the transformer creates `sc_lap_proxy` (`laptime_zscore > 0.8`) and `recent_sc_proxy` (rolling 3-lap sum of `sc_spike_flag`) specifically to capture SC information as model-friendly binary/count signals.

6.2 Cumulative_Degradation Negatives

85.6% of rows have negative Cumulative_Degradation. This is by design: the feature resets to a large negative on the pit lap (fresh tyre degradation starts at 0 and goes negative as the compound wears in). The transformer already normalises this via `deg_per_lap` and `deg_accel` (which use clipped TyreLife as the denominator).

Strategy: Yeo-Johnson handles negative values natively (unlike Box-Cox). Winsorization at 0.1% clips the most extreme negative resets without affecting the majority of values.

6.3 urgency_index Spikes

`urgency_index = tyre_life_norm / laps_remaining_norm`. As `laps_remaining_norm` $\rightarrow$ 0 (end of race), this ratio explodes. The transformer clips the denominator at 0.01 to prevent infinity, but values can still reach 70–100.

Strategy: WinsorCapper (99.9%) caps the spike, followed by Yeo-Johnson to normalise the resulting right-skewed distribution.

7 Class Imbalance Considerations

PitNextLap is 19.9% positive — a 4:1 imbalance. This is NOT handled inside the preprocessing module for the following reasons:

- SMOTE and oversampling techniques must be applied only to training data, not to validation/test sets. Including them in a transformer that is also used for test sets would cause data leakage.
- The preprocessing module is designed to be used inside sklearn Pipelines and cross-validation loops where SMOTE would be added as a separate step after preprocessing.
- Probability calibration, `class_weight='balanced'`, and decision-threshold tuning are model-level concerns.

Recommended approach for handling imbalance (outside this module):

- XGBoost / LightGBM: set `scale_pos_weight = 351759 / 87381  $\approx$  4.03`
- Logistic Regression / SVM: use `class_weight='balanced'`

- RandomForest: use `class_weight='balanced_subsample'`
 - If oversampling is needed: apply `imblearn.over_sampling.SMOTE` inside a Pipeline after this preprocessor, fitting on `x_train_pp` only
-

8 Feature Selection Guidance (Post-Preprocessing)

The final 79-column matrix is designed for downstream feature selection before ensemble training. The following techniques are recommended:

8.1 Variance Threshold

Binary flags with near-zero variance (e.g. `cpd_WET` covering only 0.3% of data) may be removed. Recommended threshold: remove features where `variance < 0.01` (i.e. > 99% same value).

8.2 Correlation-Based Removal

Several feature pairs are intentionally related (e.g. `tyre_life_norm` and `tyre_overrun_amount`; `laps_remaining` and `laps_remaining_norm`). Check pairwise Pearson correlation after preprocessing and consider dropping one of any pair with $|r| > 0.95$.

8.3 Tree-Based Feature Importance

Fit a lightweight LightGBM with default parameters on `x_train_pp` and extract gain-based feature importances. Remove the bottom 10–15 features by importance. Key features expected to dominate: `tyre_life_norm`, `urgency_index`, `laps_to_cliff`, `cliff_proximity`, `past_cliff`, `race_phase_late`, `in_pit_window`.

8.4 Permutation Importance (Validation Set)

After initial model training, run permutation importance on `x_test_pp` with `y_test`. Features with mean permutation importance < 0 can be safely removed (they add noise).

8.5 Avoid Removing These Features

The following features capture unique signal that may be missed by simple correlation or importance metrics:

- `sc_lap_proxy` / `recent_sc_proxy` — safety car context is rare (0.73%) but strategically critical
 - `prev_pitstop` — prevents double-pit false positives; near-zero correlation with target but high precision signal
 - `is_first_stint` — carries FIA two-compound rule information that no other feature encodes
 - `is_cluster_lap` — `pit_cluster_size > 3` flags SC pit-window surges; remove only if `pit_cluster_size` covers the same signal
-

9 Model Ensemble Compatibility

Model	Binary Flags	Ordinal Ints	Scaled Cont	Notes
XGBoost / LightGBM	✓ passthrough	✓ scaled	✓ scaled	Default config; tree splits ignore scale
RandomForest	✓ passthrough	✓ scaled	✓ scaled	Scale-invariant; benefits from cleaner distributions
Logistic Regression	Set passthrough_ binary=False	✓ scaled	✓ scaled	All features should be scaled
Neural Network (MLP)	Set passthrough_ binary=False	✓ scaled	✓ scaled	Normalised inputs essential
CatBoost	✓ passthrough	✓ scaled	✓ scaled	Internally robust; scaling still helps for ensemble stacking

For stacking ensembles: use the preprocessed matrix for all base learners. The meta-learner (typically logistic regression) will receive the scaled outputs of base models, not the raw feature matrix.

10 API Reference

F1Preprocessor

Parameter	Type	Default	Description
passthrough_binary	bool	True	If True, binary flags are left as 0/1. Set False to StandardScale them for linear/neural models.
winsor_q	tuple	(0.001, 0.999)	Quantile bounds for WinsorCapper applied to HIGH_SKEW_CONT features before PowerTransformer.

Key Methods

Method	Description
fit(X, y=None)	Learn scalers, imputers, and WinsorCapper quantiles from training data. Must be called before transform().
transform(X)	Apply fitted transformations. Returns a pandas DataFrame with original column order preserved.

Method	Description
<code>fit_transform(X, y=None)</code>	Convenience: fit then transform in one call. Equivalent to <code>fit(X).transform(X)</code> .
<code>get_feature_groups()</code>	Returns dict mapping group names to their column lists. Useful for debugging and cross-referencing.
<code>check_output(X_transformed)</code>	Returns <code>describe().T</code> with added null-count column. Quick sanity check post-transform.

build_pipeline()

Parameter	Description
<code>feature_transformer</code>	An <code>F1PitStopFeatureTransformer</code> instance (configured but not yet fit)
<code>passthrough_binary</code>	Passed to <code>F1Preprocessor</code> ; see above
<code>winsor_q</code>	Passed to <code>F1Preprocessor</code> ; see above

Returns a sklearn `Pipeline([('feature_engineering', ...), ('preprocessing', ...)])` that can be used end-to-end or wrapped in cross-validation.